CancerGuard AI: Project Documentation & Architecture Report

1. Executive Summary CancerGuard AI is a secure, modern, and production-ready AI-assisted cancer risk screening and clinical assessment platform. The platform is designed to assist medical professionals by evaluating uploaded scans and providing risk assessment models for multiple types of cancer. It features secure authentication, interactive dashboards, and the ability to generate clinical-grade PDF reports with strict medical disclaimers.

> *Strict Medical Disclaimer: This AI system provides risk assessment only and is not a substitute for professional medical diagnosis. All output assessments are experimental screening guides and must be verified by a qualified physician.*

2. Key Features 1. Secure Authentication & Authorization: - JWT-based user and administrator sessions. - Encrypted password storage and role-based access control (RBAC). - Strict data isolation: General users can only access their own scans and reports.

2. Modular AI Screening Workflows:

• Skin Cancer: Analysis of dermoscopic lesions.

• Brain Tumor: Evaluation of MRI scans.

• Lung Cancer: Analysis of chest X-rays.

• Breast Cancer: Assessment of digital mammograms.

3. Interactive Dashboards:

• Patient Dashboard: Tracks risk history, metrics, and displays custom SVG risk distributions.

• Admin Portal: Aggregates platform statistics, category breakdowns, user registries, and comprehensive audit logs.

4. Clinical PDF Reporting:

• Generates on-demand, clinical-grade PDF reports using ReportLab.

• Reports include patient demographics, assessed risk levels, and legally sound medical disclaimers.

5. Containerization:

• Fully containerized with Docker and Docker Compose for seamless multi-environment deployment.

3. Technology Stack The application adopts a decoupled architecture, separating the client-side interface from the server-side API, ensuring scalability and ease of maintenance.

Frontend - Framework: Next.js (App Router) - Language: TypeScript - Styling: Tailwind CSS - Icons: Lucide React

Backend - Framework: FastAPI (Python) - Server: Uvicorn - Data Validation: Pydantic - ORM: SQLAlchemy - PDF Generation: ReportLab

Database - Primary: PostgreSQL - Fallback/Testing: SQLite (automatic fallback)

Infrastructure - Containerization: Docker & Docker Compose - Version Control: Git & GitHub

4. Project Structure The repository is logically divided into backend and frontend ecosystems.

```
CancerGuardAI/
■■■ backend/
■   ■■■ app/
■   ■   ■■■ models/        # Database models (User, Scan, Prediction, Report)
■   ■   ■■■ routes/        # Router endpoints (auth, predict, scans, reports, admin)
■   ■   ■■■ services/      # AI models & PDF Report generator services
■   ■   ■■■ utils/         # Helper functions (security, db initialization)
■   ■   ■■■ config.py      # App configurations
■   ■   ■■■ main.py        # Application entrypoint
■   ■■■ Dockerfile
■   ■■■ requirements.txt
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ app/           # App routes (dashboard, login, upload, result)
■   ■   ■■■ components/    # Reusable UI widgets and layout wrappers
■   ■   ■■■ context/       # Authentication context provider
■   ■   ■■■ services/      # API wrappers (Axios/Fetch)
■   ■■■ Dockerfile
■   ■■■ package.json
■■■ docker-compose.yml
```

5. Security & Data Privacy - Encryption: Passwords are mathematically hashed before storage. - Session Security: JSON Web Tokens (JWT) are used to securely transmit information between the client and server. - Access Control: The system distinguishes between standard `User` accounts and `Admin` accounts (`is_admin=True`). Administrators have aggregate visibility, while patients are restricted to their personal health records.

6. Deployment Architecture The platform is designed to be deployed using Docker Compose, orchestrating three primary services: 1. Frontend Service: Next.js application running on port `3000`. 2. Backend Service: FastAPI application running on port `8000`. 3. Database Service: PostgreSQL database handling persistent storage.

To initialize the entire stack:

`docker-compose up --build`

This ensures that the development, staging, and production environments remain perfectly consistent.